MODULE 18

Authentication and Security

Threat Modeling, OAuth 2.0, OpenID Connect, API Authorization, and Mobile App Hardening

What is Covered in This Module

Part 1

Identity and Tokens

Mobile threat modeling, OAuth 2.0 with PKCE, OpenID Connect and JWT validation, refresh token rotation and revocation

Part 2

Securing the App and the API

API authorization, step-up authentication, biometrics and passkeys, certificate pinning, attestation, security review workshop



©Logical Imagination Group LLC

PART 1

Identity: Threats, OAuth 2.0, OpenID Connect, and Tokens

1 Mobile Threat Modeling and Security Standards

2 OAuth 2.0 for Mobile: Authorization Code with PKCE

3 OpenID Connect and JWT Validation

4 Token Lifecycle: Refresh Rotation, Revocation, and Sign-Out

Lesson 1: Mobile Threat Modeling and Security Standards

- Explain why a mobile app is an untrusted client and what that means for design
- Build a threat model of the sign-in and transfer flows using STRIDE
- Navigate the OWASP MASVS control groups and the OWASP API Security Top 10
- Rank threats by impact and likelihood and choose controls for the top ones



©Logical Imagination Group LLC

The App Is an Untrusted Client

- Anything shipped inside the app can be extracted and read
- The device may be rooted, jailbroken, or running malware
- The network may be hostile: public Wi-Fi, proxies, interception
- Therefore the server decides; the client only asks

Trusted	Untrusted
server-side authorization	the app binary and its resources
server-side validation	local storage on the device
signing keys in an HSM	anything the app sends
	the network path



©Logical Imagination Group LLC

Threat Modeling with STRIDE

- STRIDE: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
- Rank each threat by impact and likelihood
- Pick controls for the top threats and record what you accepted

Threat	Description
Spoofing	stolen refresh token used from another phone
Tampering	amount changed by a proxy on public Wi-Fi
Repudiation	customer denies sending; need an audit record with auth method
Info disclosure	token or account number in a log line or crash report
Denial	flood of login attempts locks real customers out
Elevation	customer A reads customer B's account by changing the id



©Logical Imagination Group LLC

Standards That Give You a Checklist

- OWASP MASVS groups mobile controls: storage, crypto, auth, network, platform, code, resilience, privacy
- OWASP MASTG describes how to test each control
- OWASP API Security Top 10 covers object, function, and property level authorization
- RFC 9700 is the current best practice for OAuth 2.0 security



©Logical Imagination Group LLC

Sample Checklist Items

MASVS Group	Example Question
STORAGE	is the refresh token encrypted with a hardware-backed key?
CRYPTO	Do we use platform crypto, with no hard-coded keys?
AUTH	Does the server check scope and ownership on every call?
NETWORK	TLS only? Pinning decision recorded?
PLATFORM	Are exported components and deep links validated?
CODE	Are dependencies scanned and AI-generated code reviewed?
RESILIENCE	What is the response to a rooted device or tampering?
PRIVACY	Do logs and analytics exclude personal data?



©Logical Imagination Group LLC

Lesson 1

Skill Check

1. Why must security decisions be made on the server?

A) Servers are faster
B) The app binary, device, and network are controlled by the attacker
C) Phones have no CPU
D) It reduces app size

2. What does the S in STRIDE stand for?

A) Storage
B) Spoofing
C) Sessions
D) Signing

3. Which API weakness lets a caller read another customer's object by changing an id?

A) Broken object level authorization
B) Rate limiting
C) Cross-site scripting
D) Weak TLS

4. Why record an owner and date for an accepted risk?

A) To assign blame
B) So the acceptance is a managed decision that is reviewed
C) It is optional
D) To satisfy a compiler

©Logical Imagination Group LLC

Lesson 2: OAuth 2.0 for Mobile: Authorization Code with PKCE

- Explain the roles in OAuth 2.0 and why a mobile app is a public client
- Walk through the authorization code flow with PKCE step by step
- Choose a safe redirect strategy and an external user agent
- Implement code verifier and challenge generation, URL building, and redirect validation



©Logical Imagination Group LLC

Roles and Why the App Never Sees the Password

- OAuth 2.0 lets an app act with limited access without handling the user's password
- Roles: resource owner, client (the app), authorization server, resource server (the API)
- A mobile app is a public client: it cannot keep a secret
- The user signs in at the authorization server, not inside the app

```
Resource owner (customer)      Client (PNC Mobile app)
        |                              |
        | signs in at                  | asks for scope accounts:read transfers:create
        v                              v
Authorization server  -- tokens -->  Client  -- access token -->  Resource server (API)

The password goes to the authorization server only.
```



©Logical Imagination Group LLC

Authorization Flow with PKCE

- The app creates a random code verifier and sends only its hash, the challenge
- The browser returns a short-lived authorization code to the app
- The app exchanges the code plus the verifier for tokens
- A stolen code is useless without the verifier



©Logical Imagination Group LLC

Authorization Code with PKCE

```
const verifier  = generateCodeVerifier();            // 32 random bytes, base64url, 43 chars
const challenge = codeChallengeS256(verifier);       // BASE64URL(SHA256(verifier))

const url = buildAuthorizationUrl({
  authEndpoint, clientId, redirectUri, scopes,
  state, nonce, codeChallenge: challenge        // sent in leg 1
});

// leg 2, after the redirect returns ?code=...&state=...
const { code } = parseRedirect(redirectUrl, redirectUri, state);
await postToken(tokenEndpoint, { grant_type: "authorization_code", code, code_verifier:
verifier, /* ... */ });
```



©Logical Imagination Group LLC

Redirects, State, and the External User Agent

- Use the system browser or the platform authentication session, never an embedded WebView
- Prefer a verified https redirect (App Links or Universal Links) over a custom scheme
- Match the redirect URI exactly, not by prefix
- The state value ties the response to the request and defeats forged redirects

Good	https://app.pnc.example/oath/callback	verified App Link or Universal Link
OK	com.pnc/mobile:/oauth2redirect	private-use scheme in reverse domain form (needs PKCE)
Bad	myapp://callback	short scheme, easy to claim
Bad	embedded WebView login	app sees the keystrokes



©Logical Imagination Group LLC

Lesson 2

Skill Check

1. What attack does PKCE defend against?

A) SQL injection
B) Interception of the authorization code by another app
C) Weak passwords
D) Slow networks

2. Why is a mobile app a public client?

A) It is free
B) It cannot keep a client secret because the binary is distributed to everyone
C) It has no network
D) It uses HTTP

3. Why use the system browser instead of an embedded WebView?

A) It is prettier
B) The app should never see the user's credentials or cookies
C) WebViews are slower
D) Browsers cache tokens

4. What is wrong with matching a redirect URI by prefix?

A) Nothing
B) An attacker can append a path or use a look-alike host
C) It is slower
D) It breaks logging



©Logical Imagination Group LLC

Lesson 3: OpenID Connect and JWT Validation

- Explain what OpenID Connect adds to OAuth 2.0 and distinguish ID tokens from access tokens
- Describe discovery, the JWKS document, and key rotation
- List every check required to validate a JWT and the order in which to do them
- Implement a validator that rejects the common JWT attacks



©Logical Imagination Group LLC

OpenID Connect: Who Signed In

- OIDC is an identity layer on top of OAuth 2.0
- Requesting the openid scope returns an ID token
- The ID token says who authenticated, how, and when; it is for the client
- The access token is for the API and the app should treat it as opaque

Code Bite

```
// ID token claims
{
  "iss": "https://login.pnc.example",
  "sub": "cust_8f31",
  "aud": "pnc-mobile",
  "exp": 1800000300,
  "iat": 1800000000,
  "nonce": "n-0S6_WzA2Mj",
  "auth_time": 1799999990,
  "amr": ["pwd", "bio"]
}
```



©Logical Imagination Group LLC

Keys, Discovery, and Rotation

- The issuer signs tokens with a private key and publishes the public keys as a JWKS
- The token header names the key with kid
- Fetch the JWKS over https from the discovery document and cache it
- Refetch once when a kid is unknown, because keys rotate



©Logical Imagination Group LLC

Keys, Discovery, and Rotation

```
GET https://login.pnc.example/.well-known/openid-configuration
{
  "issuer": "https://login.pnc.example",
  "authorization_endpoint": "https://login.pnc.example/authorize",
  "token_endpoint": "https://login.pnc.example/token",
  "jwks_uri": "https://login.pnc.example/keys"
}
```

```
GET https://login.pnc.example/keys
{ "keys": [ { "kty": "RSA", "kid": "k1", "alg": "RS256", "use": "sig", "n": "...", "e":
"AQAB" } ] }
```



©Logical Imagination Group LLC

Validating a JWT: The Checks and Their Order

- Choose the algorithm from your allow list, never from the token
- Verify the signature before trusting any claim
- Check issuer, audience, expiry, not-before, and nonce
- Reject alg none and algorithm confusion



©Logical Imagination Group LLC

Validating a JWT: The Checks and Their Order

```
const alg = header.alg;


if (!allowed.includes(alg)) throw new JwtError("alg_not_allowed", ...);
// none, HS256 confusion


const ok = candidates.some(jwk => cryptoVerify("RSA-SHA256", signingInput, createPublicKey({
key: jwk, format: "jwk" }), signature));
if (!ok) throw new JwtError("bad_signature", ...);


// only now read the claims
if (claims.iss !== opts.issuer) throw ...;
if (!audienceMatches(claims.aud, opts.audience)) throw ...;
if (opts.nowSec - skew >= claims.exp) throw ...;
if (opts.nonce !== undefined && claims.nonce !== opts.nonce) throw ...
```



©Logical Imagination Group LLC

Lesson 3

Skill Check

1. What is the ID token for?

A) Calling the API
B) Telling the client who authenticated and how
C) Refreshing access
D) Encrypting traffic

2. Where may the validator get the signing algorithm from?

A) The token header
B) Its own allow list for the issuer
C) The request URL
D) The user

3. What must happen before claims are trusted?

A) Nothing
B) The signature is verified
C) The token is decoded twice
D) The token is logged

4. What does the nonce protect against?

A) Weak passwords
B) Replay of an ID token issued for an earlier request
C) Slow networks
D) Expired certificates



©Logical Imagination Group LLC

Lesson 4: Token Lifecycle: Refresh Rotation, Revocation, and Sign-Out

- Explain why access tokens are short lived and refresh tokens are protected more carefully
- Describe refresh token rotation and reuse detection and what each defends against
- Handle the lost response problem without weakening theft detection
- Implement a server-side refresh token store with families, lifetimes, and revocation



©Logical Imagination Group LLC

Short Access Tokens and Long Refresh Tokens

- Access tokens live minutes; a leak is brief
- Refresh tokens live days or weeks and let the app stay signed in
- The refresh token is the valuable secret: store it in hardware-backed storage
- Idle and absolute lifetimes bound a login even when the token is never stolen

Token Type	Lifetime	
access token	5 minutes	memory only on the device; sent to the API
refresh token	30 days idle 90 days absolute	Keystore or Keychain; sent only to the token endpoint
ID token	once	read at sign-in, then discarded or kept only for the claims needed



©Logical Imagination Group LLC

Rotation and Reuse Detection

- Every refresh returns a new refresh token and retires the old one
- Tokens belong to a family that starts at one login
- Presenting a retired token is a sign of theft: revoke the whole family
- Store only a hash of each token on the server

Code Bite

```
login              -> family F1: T1 (active)
refresh with T1    -> T1 rotated, T2 active
refresh with T2    -> T2 rotated, T3 active

attacker replays T1 (rotated)  ->

REUSE DETECTED
revoke family F1: T3 revoked too
everyone with F1 must sign in again
```



©Logical Imagination Group LLC

The Lost Response Problem and Sign-Out

- A rotated token can be presented twice by an honest app if the response was lost
- A short grace window tolerates one retry without weakening theft detection much
- Sign-out must revoke on the server, not only delete the token on the phone
- Revoking an unknown token returns success so logout leaks nothing

Code Bite

```
if (entry.status === "rotated") {
  if (graceMs > 0 && now -
            entry.rotatedAt <= graceMs) {
     return mint(entry.familyId, ...);
     // lost response: tolerate a retry
     // inside the window
  }
  revokeFamily(entry.familyId);
  // outside the window: treat as theft
  throw new RefreshError("reuse_detected");
}
```



©Logical Imagination Group LLC

Lesson 4

Skill Check

1. Why store only a hash of a refresh token?

A) To save space
B) A database leak does not expose working tokens
C) Hashes are faster
D) It is required by HTTP

2. A retired refresh token is presented again outside the grace window. What happens?

A) It is accepted
B) The whole family is revoked
C) Only that token is revoked
D) Nothing

3. Why does sign-out call the server?

A) To clear the screen
B) Deleting the token on the phone does not stop a copy
C) Servers need analytics
D) To refresh the token

4. Why add a short grace window?

A) To speed up refresh
B) An honest app may retry after a lost response
C) To extend the token life
D) To skip hashing



©Logical Imagination Group LLC